

Aula: Introdução ao MongoDB para Alunos de Banco de Dados Relacional

Carga horária: 4 horas **Pré-requisito:** Conhecimento prévio de modelagem relacional (SQL, normalização, chaves primárias/estrangeiras) **Formato:** Teoria + Demonstração + Prática guiada

Sobre esta aula

Persona do instrutor: Especialista em Bancos de Dados (Relacionais e NoSQL), com foco em ajudar profissionais vindos do mundo SQL a migrarem seu raciocínio para modelagem orientada a documentos, sem perder o rigor técnico que já têm.

O objetivo aqui **não é convencer ninguém de que MongoDB é "melhor"** que um banco relacional — é mostrar que são ferramentas com propósitos diferentes, e que entender *quando* usar cada uma é a habilidade mais valiosa de um profissional de dados.

Sumário

1. Módulo 1 — Do mundo relacional ao mundo de documentos (60 min)
 2. Módulo 2 — Modelagem e BSON na prática (60 min)
 3. Módulo 3 — Passo a passo: criando um banco MongoDB online (60 min)
 4. Módulo 4 — CRUD, consultas e agregações na prática (60 min)
 5. Exercícios finais
 6. Referências e próximos passos
-

Módulo 1 — Do mundo relacional ao mundo de documentos (60 min)

1.1 Por que existe o MongoDB?

Bancos relacionais (PostgreSQL, MySQL, SQL Server) foram desenhados nos anos 1970 para um cenário de **dados estruturados, esquema fixo e forte consistência**. Eles continuam sendo a escolha certa para a maioria dos sistemas transacionais (financeiro, ERP, e-commerce com estoque).

O MongoDB nasceu nos anos 2000 para resolver problemas que o modelo relacional torna caros:

- Dados com **estrutura variável** (cada "registro" tem campos diferentes)
- **Alto volume de escrita** distribuído em muitos servidores (escalabilidade horizontal)

- Dados que são naturalmente **hierárquicos/aninhados** (um pedido com itens, um post com comentários)
- Times que precisam **iterar rápido** sem rodar migration a cada mudança de schema

1.2 De tabelas para coleções: o "tradutor" mental

Conceito Relacional (SQL)	Conceito MongoDB	Observação
Banco de dados ((DATABASE))	Banco de dados ((database))	Mesmo conceito
Tabela ((TABLE))	Coleção ((collection))	Coleção não exige schema fixo
Linha / Registro ((ROW))	Documento ((document))	Documento = objeto JSON/BSON
Coluna ((COLUMN))	Campo ((field))	Campos podem variar entre documentos
Chave primária ((PRIMARY KEY))	((_id))	Gerado automaticamente se não for informado
((JOIN))	((lookup)) ou embutir dados	MongoDB prefere embutir a relacionar
((SCHEMA)) rígido	Schema flexível (ou validado via regras)	Flexível não é sinônimo de "sem regra"

1.3 A maior mudança de raciocínio: normalizar vs embutir

No relacional, vocês aprenderam a **normalizar** para evitar redundância. No MongoDB, a pergunta muda: "**esses dados são sempre lidos juntos?**" Se sim, geralmente faz sentido **embutir** (nested document) em vez de separar em coleções e usar ((lookup)).

Exemplo relacional clássico (normalizado):

```
sql
```

```
-- Tabela de pedidos
CREATE TABLE pedidos (
  id SERIAL PRIMARY KEY,
  cliente_id INT REFERENCES clientes(id),
  data_pedido DATE
);

-- Tabela de itens do pedido (1:N)
CREATE TABLE itens_pedido (
  id SERIAL PRIMARY KEY,
  pedido_id INT REFERENCES pedidos(id),
  produto VARCHAR(100),
  quantidade INT,
  preco_unitario DECIMAL(10,2)
);
```

Equivalente em MongoDB (documento único, dados embutidos):

```
javascript

// Um único documento representa o pedido inteiro,
// incluindo os itens, sem necessidade de JOIN para lê-lo
{
  "_id": ObjectId("665f1a2b3c4d5e6f7a8b9c0d"),
  "cliente_id": ObjectId("665f1a2b3c4d5e6f7a8b9c01"),
  "data_pedido": ISODate("2026-06-15"),
  "itens": [
    { "produto": "Teclado mecânico", "quantidade": 1, "preco_unitario": 350.00 },
    { "produto": "Mouse sem fio", "quantidade": 2, "preco_unitario": 89.90 }
  ]
}
```

Ponto de atenção pedagógico: isso não significa "nunca relacionar". Se os itens crescem sem limite (ex: milhões de eventos de log por pedido) ou são reutilizados por muitos documentos (ex: catálogo de produtos compartilhado), referenciar com `$lookup` continua sendo a escolha certa. Regra prática: **dados que crescem juntos e são lidos juntos → embutir. Dados reutilizados ou que crescem sem limite → referenciar.**

Módulo 2 — Modelagem e BSON na prática (60 min)

2.1 O que é BSON?

MongoDB armazena documentos em **BSON** (Binary JSON) — uma versão binária do JSON com tipos extras (datas, números decimais de precisão, ObjectId, etc.). Para quem

programa, isso significa: você manipula objetos JavaScript/JSON normalmente, e o driver converte para BSON por baixo dos panos.

javascript

```
// Exemplo de documento BSON – visualmente é JSON,  
// mas internamente tem tipos como ObjectId e ISODate  
{  
  "_id": ObjectId("665f1a2b3c4d5e6f7a8b9c0d"), // tipo especial, gerado automaticamente  
  "nome": "Karize", // string  
  "ativo": true, // boolean  
  "idade": 32, // int32  
  "salario": NumberDecimal("4500.50"), // decimal de precisão exata  
  "criado_em": ISODate("2026-06-30T10:00:00Z") // data real, não string  
}
```

2.2 Schema flexível ≠ sem regras

Um erro comum de quem vem do relacional é achar que MongoDB "não tem regra nenhuma". Na prática, usa-se **Schema Validation** para impor regras mínimas mesmo sem um schema rígido como o SQL:

javascript

```
// Criando uma coleção com regras de validação,  
// parecido com um CHECK CONSTRAINT do SQL  
db.createCollection("alunos", {  
  validator: {  
    $jsonSchema: {  
      bsonType: "object",  
      required: ["nome", "email", "matricula"], // campos obrigatórios  
      properties: {  
        nome: {  
          bsonType: "string",  
          description: "deve ser uma string e é obrigatório"  
        },  
        email: {  
          bsonType: "string",  
          pattern: "^.+@.+\\.\\.+$", // validação simples de formato de e-mail  
          description: "deve ser um e-mail válido"  
        },  
        matricula: {  
          bsonType: "int",  
          minimum: 1,  
          description: "deve ser um inteiro positivo"  
        }  
      }  
    }  
  }  
});
```

2.3 Exercício rápido (10 min, em grupo)

Peça aos alunos para pegar um modelo relacional que já conhecem (ex: `clientes` + `pedidos` + `itens_pedido`) e desenhem, no papel ou quadro, como ficaria embutido em MongoDB. Discuta em conjunto os trade-offs.

Módulo 3 — Passo a passo: criando um banco MongoDB online (60 min)

Aqui vamos usar o **MongoDB Atlas**, o serviço gerenciado oficial do MongoDB (existe um tier gratuito, suficiente para fins didáticos).

Passo 1 — Criar a conta

1. Acesse <https://www.mongodb.com/cloud/atlas/register>
2. Cadastre-se com e-mail/senha ou login Google/GitHub
3. Confirme o e-mail, se solicitado

1. No painel do Atlas, clique em **"Create"** (ou "Build a Database")
2. Escolha a opção **M0 Free** (cluster compartilhado, gratuito, ideal para estudo)
3. Selecione um provedor de nuvem (AWS, Azure ou GCP — qualquer um serve) e a região mais próxima (ex: São Paulo (`sa-east-1`))
4. Dê um nome ao cluster (ex: (`cluster-aula-mongodb`))
5. Clique em **"Create Deployment"**

Passo 3 — Configurar acesso (usuário e segurança)

1. Em **"Security Quickstart"**, crie um usuário de banco de dados:
 - Username: (`aluno_admin`)
 - Password: gere uma senha forte (clique em "Autogenerate Secure Password" e **salve em local seguro**)
2. Em **"Where would you like to connect from?"**, escolha:
 - Para fins de aula/estudo: **"My Local Environment"** e clique em **"Add My Current IP Address"**
 - Alternativa rápida (apenas para ambientes de teste, **nunca em produção**): adicionar (`0.0.0.0/0`) para liberar todos os IPs

Passo 4 — Obter a string de conexão

1. No painel, clique em **"Connect"** no seu cluster
2. Escolha **"Drivers"** (se for conectar via código) ou **"MongoDB Shell"** (se for usar (`mongosh`))
3. Copie a *connection string*, que terá este formato:

```
mongodb+srv://aluno_admin:<password>@cluster-aula-mongodb.xxxxx.mongodb.net/?
retryWrites=true&w=majority
```

Substitua (`<password>`) pela senha criada no Passo 3. **Nunca** versione essa string com a senha em texto puro em repositórios públicos — use variáveis de ambiente.

Passo 5 — Conectar via terminal (mongosh)

```
bash
```

```
# Instalando o mongosh (shell oficial do MongoDB), caso ainda não tenha
# No Windows: baixar o instalador em mongodb.com/try/download/shell
# No Linux/Mac, pode-se usar o gerenciador de pacotes:
brew install mongosh # macOS com Homebrew

# Conectando ao cluster criado no Atlas
mongosh "mongodb+srv://cluster-aula-mongodb.xxxxx.mongodb.net/" --username aluno_admin
# Será solicitada a senha configurada no Passo 3
```

Passo 6 — Conectar via Node.js (driver oficial)

```
javascript

// Instalar o driver oficial antes de rodar este código:
// npm install mongodb

const { MongoClient } = require("mongodb");

// String de conexão obtida no Atlas – em projetos reais,
// isso deve vir de uma variável de ambiente (process.env), nunca hardcoded
const uri = "mongodb+srv://aluno_admin:<password>@cluster-aula-mongodb.xxxxx.mongodb.net/";

const client = new MongoClient(uri);

async function conectar() {
  try {
    await client.connect(); // abre a conexão com o cluster
    console.log("Conectado ao MongoDB Atlas com sucesso!");

    const db = client.db("escola"); // seleciona (ou cria) o banco "escola"
    const colecao = db.collection("alunos"); // seleciona (ou cria) a coleção "alunos"

    const resultado = await colecao.find().limit(5).toArray(); // busca até 5 documentos
    console.log(resultado);

  } finally {
    await client.close(); // sempre fechar a conexão ao terminar
  }
}

conectar();
```

Passo 7 — Conectar via interface gráfica (opcional)

Para alunos que preferem uma interface visual semelhante ao pgAdmin/MySQL

Workbench, instale o **MongoDB Compass** (gratuito, em mongodb.com/products/compass) e cole a mesma *connection string* do Passo 4.

Módulo 4 — CRUD, consultas e agregações na prática (60 min)

Todos os exemplos abaixo podem ser executados no `mongosh` conectado ao cluster criado no Módulo 3.

4.1 Create (Inserir)

```
javascript

// Selecionando/criando o banco "escola"
use escola

// Inserindo um único documento na coleção "alunos"
// (equivalente a um INSERT INTO no SQL)
db.alunos.insertOne({
  nome: "Maria Silva",
  email: "maria.silva@email.com",
  matricula: 1001,
  curso: "Gestão da Informação",
  ativo: true,
  notas: [8.5, 7.0, 9.2] // array embutido – não existe equivalente direto em uma coluna
});

// Inserindo vários documentos de uma vez
// (equivalente a múltiplos INSERTs, mas em uma única operação)
db.alunos.insertMany([
  { nome: "João Pedro", email: "joao.pedro@email.com", matricula: 1002, curso: "Banco de Dados", ativo: true },
  { nome: "Ana Costa", email: "ana.costa@email.com", matricula: 1003, curso: "Banco de Dados", ativo: false }
]);
```

4.2 Read (Consultar)

```
javascript
```

```
// Buscar todos os documentos da coleção
// (equivalente a SELECT * FROM alunos)
db.alunos.find();

// Buscar com filtro – apenas alunos ativos
// (equivalente a SELECT * FROM alunos WHERE ativo = true)
db.alunos.find({ ativo: true });

// Buscar com filtro e projeção (escolher quais campos retornar)
// (equivalente a SELECT nome, email FROM alunos WHERE curso = 'Banco de Dados')
db.alunos.find(
  { curso: "Banco de Dados" },      // filtro
  { nome: 1, email: 1, _id: 0 }    // projeção: inclui nome e email, exclui _id
);

// Operadores de comparação – alunos com média de notas acima de 8
// $avg é usado dentro de agregação; aqui usamos $gte como exemplo simples de filtro
db.alunos.find({ notas: { $elemMatch: { $gte: 8 } } }); // ao menos uma nota >= 8

// Ordenando resultados por matrícula em ordem decrescente
// (equivalente a ORDER BY matricula DESC)
db.alunos.find().sort({ matricula: -1 });
```

4.3 Update (Atualizar)

javascript

```
// Atualizando um único documento
// (equivalente a UPDATE alunos SET curso = 'Ciência de Dados' WHERE matricula = 1002)
db.alunos.updateOne(
  { matricula: 1002 },           // filtro: qual documento atualizar
  { $set: { curso: "Ciência de Dados" } } // operação: define novo valor do campo
);

// Atualizando vários documentos de uma vez
// (equivalente a UPDATE alunos SET ativo = false WHERE curso = 'Banco de Dados')
db.alunos.updateMany(
  { curso: "Banco de Dados" },
  { $set: { ativo: false } }
);

// Adicionando um valor a um array existente, sem sobrescrever o array inteiro
// (não existe equivalente direto no SQL tradicional sem tabela auxiliar)
db.alunos.updateOne(
  { matricula: 1001 },
  { $push: { notas: 10.0 } } // adiciona 10.0 ao array "notas"
);
```

4.4 Delete (Remover)

javascript

```
// Removendo um único documento
// (equivalente a DELETE FROM alunos WHERE matricula = 1003)
db.alunos.deleteOne({ matricula: 1003 });

// Removendo vários documentos que atendem a um critério
// (equivalente a DELETE FROM alunos WHERE ativo = false)
db.alunos.deleteMany({ ativo: false });
```

4.5 Agregações — o "GROUP BY" do MongoDB

javascript

```
// Pipeline de agregação: calcula a média de notas por curso
// (equivalente a SELECT curso, AVG(media_notas) FROM alunos GROUP BY curso)
db.alunos.aggregate([
  // Etapa 1: calcula a média de cada array "notas" por documento
  {
    $project: {
      curso: 1,
      media_individual: { $avg: "$notas" }
    }
  },
  // Etapa 2: agrupa por curso e tira a média geral
  {
    $group: {
      _id: "$curso",
      media_geral: { $avg: "$media_individual" },
      total_alunos: { $sum: 1 } // conta quantos documentos entraram no grupo
    }
  },
  // Etapa 3: ordena do curso com maior média para o menor
  {
    $sort: { media_geral: -1 }
  }
]);
```

Exercícios finais

Para os alunos resolverem individualmente ou em duplas, usando o cluster criado no Módulo 3:

1. Crie uma coleção `cursos` com pelo menos 3 documentos contendo `nome`, `carga_horaria` e `instrutor`.
2. Modele um documento de `aluno` que **embute** os cursos em que ele está matriculado (em vez de referenciar). Justifique a escolha.
3. Escreva uma query que liste apenas os alunos com `media_geral` (calculada via agregação) maior que 7.
4. Escreva uma `updateMany` que marque como `ativo: false` todos os alunos de um curso específico.
5. **Desafio:** modele o mesmo cenário usando `$lookup` (referenciando `cursos` em vez de embutir) e compare as duas abordagens — quando cada uma seria mais adequada?

Referências e próximos passos

- Documentação oficial: <https://www.mongodb.com/docs/>
 - MongoDB University (cursos gratuitos): <https://learn.mongodb.com/>
 - Próximo tema sugerido: índices em MongoDB e comparação de performance com índices SQL
 - Próximo tema sugerido: replicação e sharding (escalabilidade horizontal)
-

Material produzido para uso didático — sinta-se à vontade para adaptar a duração e a profundidade de cada módulo conforme o ritmo da turma.